



Tutorial

Sanya Bhatia

The background is a dark navy blue with various white and light blue geometric elements. There are thin white lines forming a network-like pattern, some horizontal bars, and a small circle. A light blue line with a circle at its end is on the left. A series of vertical lines is at the top center. A light blue circle is at the bottom right. A light blue line with a circle at its end is on the right side.

▶ **WHAT IS DATALOG?**

01

► WHAT IS DATALOG?

Datalog is a declarative query language, derived from Prolog. The premise is similar to that of SQL.

Its logic is based on first order logic



▶ HOW DOES IT WORK?

02

► HOW DOES IT WORK?

It works by using predicate patterns :[(predicate identifier)]

And

Data Patterns: [<entity> <attribute> <value>]

A basic query looks like:

```
[ :find <entity>
  :where
  [<entity> <attribute> <value>]]
```

*The **:find** statement will retrieve all the entity that satisfy the data pattern defined in the **:where** clauses*

► HOW IT WORKS II

Joining entities

```
[ :find ?name
  :where
  [?person :first-name ?name]
  [?person :lives-in ?city]
  [?city :located-in France]]
```

```
[(lives-in-country ?person ?country)
 [?person :lives-in ?city]
 [?city :located-in ?country]]
```

Joining can also be used a separate rule,
For example:

```
[ :find ?name
  :where
  [?person :first-name ?name]
  (lives-in-country ?person France)]
```

This says *if a person lives in city and the city is in a country, then the rule made is “lives in the country”* and we can use that find a person that lives in a country

Contains : *This states that a person lives in a whitelisted country only if the city is paris or Lyon, it add constraints*

```
[(lives-in-country-whitelisted ?person ?country)
 [?person :lives-in ?city]
 [?city :located-in ?country]
 [{contains? #{"Paris","Lyon"} ?city}]]
```


03

► ADVANCED EXAMPLE — SQL vs DATALOG

For example: Defining a *Person*

SQL	DATALOG
<pre>CREATE TABLE person (First_name CHARVAR(50), Last__name CHARVAR(50));</pre>	<pre>[(person ?person) [?person :person/first-name ?f] [?person :person/last-name ?l] [(string? ?f)] [(string? ?l)]]</pre>
<pre>select from person p1 inner join person p2 (p1.parentOf = p2.id) where person.id = person_id;</pre>	<pre>[:find [?p1 ?p2] :where [?p1 :person/id ?person] (parent-of ?person ?p2)] ;; rule definition [(parent-of ?person ?parent) [?p1 :person/id ?person] [?parent :parentOf ?p1]]</pre>

► ADVANCE EXAMPLE — RECURSION

```
[:find [?person ?great-grand-parent]
```

```
:where
```

```
(genealogy ?person ?parent)
```

→ This is a recursive function

```
(genealogy ?parent ?grand-parent)
```

```
(genealogy ?grand-parent ?great-grand-parent)]
```

```
[(genealogy ?person ?ancestor)
```

→ Base Definition

```
(parent-of ?person ?ancestor)]
```

→ Base case: ancestor is the parent of a person

```
[(genealogy ?person ?ancestor)
```

→ Recursive definition

```
(parent-of ?person ?ancestor-1)
```

→ Takes the ancestor-1, and finds its ancestor

```
(genealogy ?ancestor-1 ?ancestor)]
```

► RESOURCES

Did you like the resources on this template? Get them for free at our other websites:

VECTORS

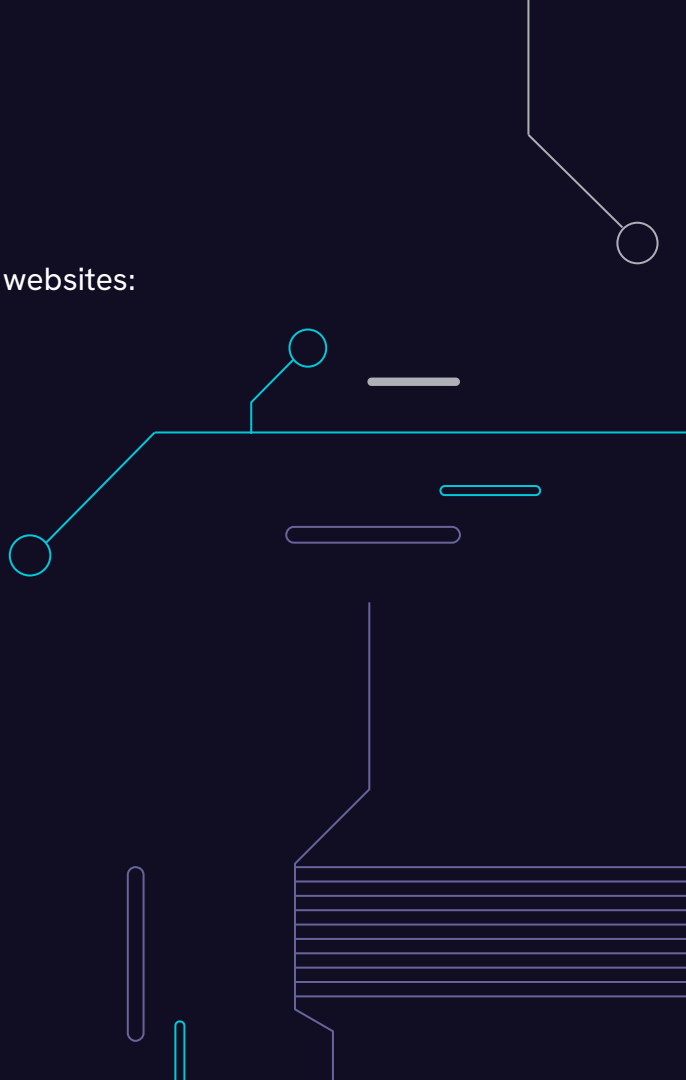
- [Artificial intelligence concept twitter header](#)
- [Artificial intelligence concept youtube thumbnail](#)

PHOTOS

- [Woman wearing virtual reality simulator](#)
- [Medium shot woman holding smartphone](#)

ICONS

- [Icon Pack: Innovation | Lineal](#)





► Conclusion

→ Alternative to SQL - Could Be!

- it has cleaner and simpler semantics
- Rules can be combined and reused in various queries
- handles complex data structures much better.

→ Limitations

- Not super compatible with newer devices
- Performance issues
- Not Heard about

Overall it is a good tool and has pros and cons, it is most useful on the Clojure dataset

CREDITS: This presentation template was created by [Slidesgo](#), and includes icons by [Flaticon](#), and infographics & images by [Freepik](#)